

WATR — Soft Calibration

Progress report & next steps · July 7, 2026

1. What was done today

1.1 Soft calibration theory

The core concept was established: m corrects for how different the balcony microclimate is from the open-field reference. The formula is:

$$ET_{crop} = m \times K_c \times ET_0$$

Soft calibration finds m by comparing how much moisture the sensor measured dropping versus how much the model expected ($K_c \times ET_0$). Taking differences (Δq) instead of absolute readings means any constant sensor offset cancels out automatically — this is the core immunity of soft calibration.

1.2 OLS regression formula understood

Instead of computing m per sample and averaging (which treats noisy low-ET days equally to reliable high-ET days), the batch formula is used:

$$m = \frac{\text{Sum}(\text{sensor_drop} \times \text{expected_drop})}{\text{Sum}(\text{expected_drop}^2)}$$

This weights each sample by the size of its `expected_drop`. Big signal days (hot, sunny, high `ET_0`) drive the result. Cloudy low-ET days barely count. The formula was derived from minimising the sum of squared errors — standard OLS.

1.3 SoftCalibrator class implemented

File: `soft_calibration/soft_calibration.py`

Method	What it does
<code>add_sample()</code>	Takes one time step of sensor + model data, computes drops, stores in buffer
<code>batch_ready()</code>	Returns True when buffer has <code>batch_size</code> samples
<code>update_m()</code>	Runs OLS on the current batch, updates m , checks convergence
<code>_check_convergence()</code>	Streak counter — resets to zero if m changes too much
<code>apply(kc, et0)</code>	Returns $ET_{crop} = m \times K_c \times ET_0$ using current m
<code>status()</code>	Returns current m , update count, convergence state as a dictionary
<code>save()</code> / <code>load()</code>	Persist m and state to JSON so calibration survives restarts

1.4 Convergence logic clarified

Convergence uses a streak counter: if $|m_{new} - m_{prev}| < \sigma$ for b consecutive batch updates, calibration is declared done. One unstable update resets the streak to zero. Parameters: `batch_size=10`, `max_updates=50`, `sigma=0.005`, `b=3`.

1.5 Presentation updated

Four new slides added to P15_WATR_Digital_Twin.pptx (slides 14-17): Hard Calibration formula, Sensor Drift chart, Soft Calibration derivation, and Calibration Update Policy table.

2. Next steps — connecting to the pipeline

2.1 Connect to query_pipeline.py

The SoftCalibrator needs to be wired into the existing data pipeline. Every time step, the pipeline must feed `add_sample()` with two things:

- Live sensor moisture reading — from the sensor group's MQTT topic
- Model predicted moisture — from the digital twin's current output

2.2 First reading edge case

On the very first time step there is no previous reading to subtract from. Do not pass `None`. Instead, store the first reading and skip it:

```
last_moisture = None # every time step: if last_moisture is not None:
cal.add_sample(moisture_prev=last_moisture, moisture_curr=current, ...)
last_moisture = current
```

2.3 Full data flow

Step	Action	Source
1	Receive sensor moisture reading	Sensor group MQTT topic
2	Receive model predicted moisture	Digital twin output
3	Call <code>add_sample()</code> with both values	<code>query_pipeline.py</code>
4	Check <code>batch_ready()</code> — if True, call <code>update_m()</code>	<code>query_pipeline.py</code>
5	Get new <code>m</code> from <code>update_m()</code>	SoftCalibrator
6	Feed new <code>m</code> back into the <code>ET_crop</code> model	Digital twin
7	Call <code>apply(kc, et0)</code> to get corrected <code>ET_crop</code>	SoftCalibrator
8	Publish new <code>m</code> and <code>ET_crop</code> to MQTT	<code>query_pipeline.py</code>
9	Call <code>save()</code> to persist state to disk	<code>query_pipeline.py</code>

2.4 MQTT topics to publish

- `watr/calibration/m` — the updated `m` value after each batch
- `watr/calibration/et_crop` — corrected `ET_crop` from `apply()`
- `watr/calibration/status` — converged flag, update count, buffer size

3. When to recalibrate — is `m` looking fishy?

3.1 Normal flow after hard calibration

Hard calibration runs once at system boot and gives you a baseline m . Soft calibration then updates m continuously in the background as seasons change. This is normal and expected — m drifting slowly over weeks is not a problem, it is the system working correctly.

3.2 The metric to watch: prediction error

```
prediction_error = sensor_reading - model_prediction
```

Random noise makes this go positive and negative randomly. A wrong m pushes it consistently in one direction:

- Consistently positive: sensor reads higher than model — m may be too low, or sensor has positive drift
- Consistently negative: model predicts higher than sensor — m may be too high

3.3 Recalibration trigger criteria

Situation	What it means	Action
$\text{avg}(\text{prediction_error}) > 5\text{mm}$ for 7 consecutive days	Systematic bias — m is wrong or sensor is drifting	Check with anomaly detection. If sensor healthy → soft recal. If sensor drifting → wait.
Sudden large jump in prediction error	Physical change — pot moved, sensor repositioned	Hard recalibration. Confirm sensor health first.
Anomaly detection group resolves a drift event	Sensor was drifting and is now confirmed healthy	Hard recalibration to re-anchor m to clean readings.
Soft cal m diverges $> 15\%$ from hard cal baseline	Large seasonal shift or something unusual	Review manually. May just be a genuine season change.
User request	User wants to re-anchor	Soft cal recommended unless sensor health is confirmed.

3.4 Decision rule summary

Anomaly detection group says sensor is healthy AND prediction error is consistently biased → recalibrate m .

Anomaly detection group flags sensor drift → do NOT recalibrate yet. Wait for them to resolve it, then hard recalibrate.